

软件工程 II

复习提纲：名词解释与简答题答案

依据《复习提纲.pdf》逐项整理，并结合本目录课程课件校正术语。

收录范围	固定名词解释、通用简答、LSP 例题、判断与修正框架
未直接作答	必须依赖现场题干的画图、接口代码、具体测试数据；相关方法已归纳为答题框架
建议用法	先背关键词，再用自己的话展开；简答题至少写定义、作用/理由、关键步骤或例子

目录

- 一、软件工程基础与历史（3 题）
- 二、项目启动、团队、质量保障与配置管理（10 题）
- 三、需求工程（11 题）
- 四、软件设计基础（3 题）
- 五、软件体系结构（10 题）
- 六、人机交互（8 题）
- 七、详细设计与对象协作（7 题）
- 八、耦合、内聚与信息隐藏（8 题）
- 九、模块化与类设计原则（13 题）
- 十、可修改性与设计模式（6 题）
- 十一、软件构造（13 题）
- 十二、软件测试（9 题）
- 十三、软件维护与演化（6 题）
- 十四、软件生命周期与过程模型（12 题）

答题提醒：题目若要求“结合实验”，请保留本答案的工程活动框架，并替换为自己项目的真实工具、规则、制品和效果。题目若要求“结合例子”，必须把原则落到具体代码或界面行为，不能只背定义。

一、软件工程基础与历史

1. 【名词解释】什么是软件工程？

标准答案：软件工程是将系统化、规范化、可度量的方法应用于软件的开发、运行和维护，并研究这些方法的学科。它用工程化活动管理复杂性和变化，使软件在质量、成本、进度与可维护性之间取得平衡。

关键词：系统化、规范化、可度量；开发、运行、维护

2. 【简答】1950s 到 2000s 的软件发展有什么阶段性特点？

- 1950s：程序规模小，以机器和硬件为中心，程序设计接近个人技艺。
- 1960s：系统规模扩大，出现软件危机，进度失控、质量低、维护困难，推动“软件工程”提出。
- 1970s：结构化分析、结构化设计、结构化程序设计和生命周期方法兴起，强调自顶向下、逐步求精与文档。
- 1980s：个人计算机普及，面向对象、软件复用、CASE 工具和质量管理发展。
- 1990s：互联网和分布式系统发展，迭代、增量、组件化、统一过程和开源协作增强。
- 2000s：敏捷方法、持续集成、Web 服务、DevOps 萌芽，强调快速反馈、持续交付和演化。

关键词：软件危机 → 结构化 → 面向对象/复用 → 互联网/迭代 → 敏捷/持续交付

3. 【简答】软件工程要解决哪些核心问题？

标准答案：软件具有复杂性、变化性、不可见性和高度协作性。软件工程通过抽象、分解、模块化、信息隐藏、过程管理、质量保障和配置管理，使系统可理解、可修改、可验证、可交付。

关键词：复杂性、变化、不可见、协作

对应材料：复习提纲第 1-2 章；课件《01-01-软件工程基础》

二、项目启动、团队、质量保障与配置管理

4. 【简答】如何管理软件团队？

标准答案：先明确共同目标、范围和完成标准，再按能力分配角色与职责；把任务分解到可跟踪粒度，建立迭代计划、例会和看板；约定沟通、代码评审、分支、测试和完成定义；持续跟踪进度、风险与质量，并通过复盘改进。管理者更应充当协调者和障碍清除者，而非只做监督。

关键词：目标、分工、计划、沟通、跟踪、风险、复盘

5. 【简答】结合课程实验，可以采取哪些团队管理办法？

标准答案：可答：使用需求/任务看板拆分工作；每人负责明确模块并设置接口负责人；用 Git 分支与 Pull Request 协作；重要设计先评审再实现；固定短会同步风险；用 CI 自动构建与测试；每轮迭代做演示和复盘。考试时应把这些办法替换或补充为自己项目中的真实做法与效果。

关键词：实验答案要写“做法 + 解决的问题 + 效果”

6. 【简答】软件团队常见的三种结构是什么？各有什么优缺点？

- 主程序员团队：决策集中，效率高、协调成本低；但关键人风险大，成员主动性可能较低，适合小型、目标明确的精英团队。
- 民主团队：成员共同决策，士气与创造性较高；但决策慢、责任边界可能模糊，适合研究或探索项目。
- 开放/混合团队：技术决策适度集中、具体工作分散协作，兼顾效率与参与度；但需要清楚角色和沟通机制，适合多数中型项目。

关键词：集中、民主、混合；结合规模和不确定性选择

7. 【名词解释】什么是软件质量保障 (SQA) ？

标准答案：软件质量保障是为保证软件过程和产品符合既定标准、规程与质量目标而开展的计划、评审、测试、审计、度量和改进活动。它既关注最终产品，也关注产生产品的过程。

关键词：过程质量 + 产品质量；预防、发现、改进

8. 【简答】一个项目的质量保障包括哪些活动？

标准答案：制定质量计划和标准；需求、体系结构、详细设计和代码评审；单元、集成、系统和验收测试；静态分析与质量度量；配置审计和发布检查；缺陷跟踪、根因分析与过程改进。评审偏向人工发现问题，测试偏向通过执行发现失效，审计检查过程和制品是否合规。

关键词：计划、评审、测试、审计、度量、改进

9. 【简答】结合实验，如何说明质量保障措施？

标准答案：示例答法：需求完成后进行交叉评审；核心设计用类图和顺序图走查；代码必须通过 Pull Request 与自动检查；关键业务编写单元测试和接口测试；CI 失败禁止合并；每次发布建立检查清单并保留版本标签；缺陷进入 Issue 跟踪并验证关闭。最后说明这些措施如何降低遗漏、回归和集成风险。

关键词：结合实验写证据：评审记录、测试、CI、Issue、发布标签

10. 【名词解释】什么是软件配置管理（SCM）？

标准答案：软件配置管理是对软件开发中的配置项进行识别、版本控制、变更控制、状态记录和配置审计的工程活动，目标是让版本可追踪、变更可控制、构建可复现、发布内容一致。Git 是工具，配置管理是完整的管理活动。

关键词：配置标识、版本控制、变更控制、状态报告、配置审计

11. 【简答】配置管理有哪些主要活动？

标准答案：配置标识：确定需要受控的代码、文档、模型、脚本和环境；版本控制：管理分支、基线和标签；变更控制：提出、评估、批准、实施和验证变更；状态报告：记录版本与变更状态；配置审计：确认发布包内容、版本和环境一致。

关键词：标识、控制、记录、审计

12. 【简答】配置管理中的变更控制过程是什么？

标准答案：提出变更请求并说明原因和范围；分析对需求、设计、代码、测试、成本和进度的影响；由负责人或变更控制委员会批准、拒绝或延期；实施变更并进行评审与回归测试；更新相关配置项、文档和基线；记录状态并审计关闭。

关键词：提出 → 影响分析 → 审批 → 实施验证 → 更新基线 → 关闭

13. 【简答】实验中如何进行配置管理？

标准答案：示例答法：将代码、需求文档、设计图和部署脚本纳入 Git；采用受保护主分支与功能分支；通过 Pull Request 审批合并；提交信息关联任务或缺陷；发布时创建 tag；CI 固化构建与测试；环境配置使用模板和密钥管理；重要变更同步更新文档。

关键词：不要只写“用了 Git”，要写规则和可追踪性

对应材料：复习提纲第 4 章；课件《01-02-项目管理基础》

三、需求工程

14. 【名词解释】什么是需求？

标准答案：需求是利益相关者为解决问题或实现目标所需要的能力、条件或约束，也是系统或系统部件必须满足的可验证规定。需求不仅包括功能，还包括质量、数据、接口和约束。

关键词：需要的能力/条件 + 系统必须满足 + 可验证

15. 【简答】业务需求、用户需求和系统级需求如何区分？

标准答案：业务需求描述组织为什么建设系统以及可度量的业务目标；用户需求描述用户借助系统完成什么任务；系统级需求从系统角度精确描述与外部实体的交互、输入输出、状态变化和约束。三者从“为什么”到“用户做什么”再到“系统必须怎样响应”逐层细化。

关键词：业务：为什么；用户：做什么；系统：如何对外交互

16. 【简答】以 ATM 为例给出三个层次的需求。

标准答案：业务需求：将校园网点人工柜台业务量降低 30%，并提供 7×24

小时基础金融服务。用户需求：客户能够使用银行卡完成取款、存款和余额查询。系统级需求：客户插卡并输入 PIN 后，系统应在验证通过时显示账户操作菜单；取款成功后更新账户余额并输出交易结果。

关键词：业务目标要可度量；系统级需求要有输入、响应和条件

17. 【简答】需求有哪些常见类型？

标准答案：功能需求描述系统做什么；数据需求描述输入、输出、存储数据及其格式和完整性；质量需求描述性能、可靠性、易用性、安全性、可维护性等；约束规定技术、法规、平台或过程限制；对外接口需求规定与用户、硬件或其他系统的交互；业务规则规定领域中的判断和计算规则。

关键词：功能、数据、质量、约束、接口、业务规则

18. 【简答】功能需求和非功能需求有什么区别？

标准答案：功能需求回答“系统做什么”，例如提交订单、查询余额；非功能需求回答“系统做得怎样”或“受什么限制”，例如 2 秒内响应、99.9% 可用、数据加密。非功能需求往往显著影响体系结构。

关键词：做什么 vs 做得怎样；非功能需求驱动架构

19. 【名词解释】什么是需求工程？

标准答案：需求工程是围绕需求获取、分析、建模、规格说明、验证和管理开展的一组活动，目标是明确为什么建、为谁建、建什么，并在变化中保持需求与后续设计、实现和测试的一致性。

关键词：获取、分析、建模、文档化、验证、管理

20. 【名词解释】什么是用例、场景和用户故事？

标准答案：用例描述参与者为达成有价值业务目标而与系统进行的一组交互；场景是用例中的一条具体执行路径；用户故事以“角色 + 功能 + 价值”简要描述用户需求，并由验收标准补充细节。

关键词：用例是一组场景；场景是一条路径；用户故事强调价值

21. 【名词解释】什么是概念类图、系统顺序图和状态图？

标准答案：概念类图描述问题域中的概念、重要属性、关系和多重性，不含方法；系统顺序图把系统看成黑箱，描述参与者与系统的消息顺序；状态图描述一个明确主体的稳定状态及事件触发的转换。

关键词：静态结构、黑箱交互、生命周期

22. 【简答】为什么需要需求规格说明？

标准答案：需求规格说明为客户、开发、测试和管理建立共同依据；减少歧义，界定范围，支持设计和测试，提供验收标准，并建立从业务目标到实现和测试的追踪关系。即使不采用厚重 SRS，也必须以用例、用户故事、接口契约等方式准确表达系统交互细节。

关键词：共同依据、减少歧义、范围、验收、追踪

23. 【简答】如何检查并修正需求中的常见错误？

标准答案：检查是否正确、完整、一致、无歧义、可行、必要、可验证、可追踪；删除“友好、快速、尽量”等不可度量词；把复合需求拆分；补充触发条件、输入输出、异常和边界；消除不同需求之间的冲突；避免把数据库、框架等实现方案写成用户需求。

关键词：正确、完整、一致、无歧义、可行、必要、可验证、可追踪

24. 【简答】如何根据功能需求设计功能测试？

标准答案：先从需求提取正常流程、替代流程、异常和业务规则；为输入划分有效/无效等价类并检查边界；明确前置条件、输入、步骤和期望结果；建立需求到测试的追踪。至少覆盖正常、边界、非法输入、权限不足和外部依赖失败。

关键词：正常、边界、异常；输入与期望结果；需求追踪

对应材料：复习提纲第 5-7 章；课件《02-01》《02-03》《02-04》《02-05》

四、软件设计基础

25. 【名词解释】什么是软件设计？

标准答案：软件设计是在需求和约束基础上，确定软件结构、构件、接口、数据、行为和实现机制的活动。它把“系统要做什么”转化为“软件如何组织和协作来实现”。

关键词：需求到实现的桥梁；结构、接口、行为、决策

26. 【简答】软件设计的核心思想是什么？

标准答案：核心是分解与抽象：把复杂系统分成职责清楚、接口稳定的模块，并隐藏容易变化的设计决策；再通过合理的职责分配和协作，使系统保持低耦合、高内聚、可修改和可验证。

关键词：分解、抽象、模块化、信息隐藏、职责协作

27. 【简答】软件设计有哪三个层次？各自关注什么？

标准答案：高层设计即体系结构设计，确定主要构件、连接、接口、部署和质量属性决策；中层设计把模块细化为子模块、类、接口及对象协作；低层设计确定方法内部的数据结构、算法、类型、语句和控制结构。详细设计通常包含中层与低层设计。

关键词：高层：架构；中层：类与协作；低层：数据结构与算法

对应材料：复习提纲第 8 章；课件《03-01-软件设计基础与体系结构概念》《04-01-详细设计》

五、软件体系结构

28. 【名词解释】什么是软件体系结构？

标准答案：软件体系结构是系统的高层结构，由构件、连接件、配置、接口、约束和关键设计决策组成，并规定这些元素如何共同满足功能与质量属性。

关键词：构件、连接、配置、接口、约束、决策

29. 【名词解释】什么是体系结构风格？

标准答案：体系结构风格是一类系统组织方式的抽象，规定允许的构件类型、连接器类型和拓扑约束。风格提供可复用的结构经验，但必须结合需求与质量属性选择。

关键词：构件词汇 + 连接器 + 约束

30. 【简答】常见体系结构风格的优缺点是什么？

- 主程序-子程序：简单高效、易理解；但中心控制强、扩展和复用较弱。
- 面向对象：封装数据与行为，支持复用和演化；但对象关系复杂时控制流难追踪。
- 分层：职责清楚、依赖受控、易维护测试；但层间调用有开销，严格分层有时不灵活。
- MVC：分离模型、视图和控制，利于多界面和并行开发；但小系统可能过度设计。
- 管道-过滤器：过滤器可复用、易组合和并行；不适合强交互或共享状态场景。
- 客户端-服务器：集中管理服务、客户端可分布；但服务器可能成为瓶颈和单点。

关键词：写“结构特点 + 优点 + 缺点 + 适用场景”

31. 【简答】体系结构设计的一般过程是什么？

标准答案：识别架构重要需求和约束；选择或组合候选风格；按功能域和职责分解构件；定义构件的供接口、需接口和连接；分配数据与部署节点；用关键场景、质量属性分析、原型和评审验证；记录决策及理由，并在迭代中演化。

关键词：需求 → 风格 → 分解 → 接口 → 部署 → 验证 → 记录

32. 【简答】包的原则有哪些？

标准答案：凝聚性原则：REP（复用/发布等价，复用粒度等于发布粒度）、CCP（共同闭包，一起变化的类放同一包）、CRP（共同复用，不应迫使客户依赖不用的类）。耦合性原则：ADP（无环依赖）、SDP（稳定依赖，依赖方向指向更稳定的包）、SAP（稳定抽象，越稳定的包应越抽象）。

关键词：REP、CCP、CRP；ADP、SDP、SAP

33. 【简答】体系结构构件的供接口和需接口是什么？

标准答案：供接口说明构件向外提供哪些能力；需接口说明构件为完成职责需要调用哪些外部能力。完整接口还应定义操作、参数、返回值、错误、协议和质量约束。需方依赖供方的抽象契约，而非具体实现。

关键词：Provided / Required；契约；依赖抽象

34. 【简答】如何为体系结构设计集成测试？

标准答案：依据构件接口和调用关系选择集成路径；验证正常交互、数据格式、错误传播、超时与外部依赖失败；采用自顶向下、自底向上或混合的增量集成；用桩替代尚未完成的被调用模块，用驱动调用被测模块；持续集成并做回归测试。

关键词：接口、调用关系、异常；增量集成；Stub/Driver

35. 【名词解释】什么是 Stub 和 Driver？

标准答案：Stub（桩）模拟被测模块所调用的下层或外部模块，按预设方式返回结果；Driver（驱动）模拟调用被测模块的上层程序，负责准备输入、触发调用并收集结果。自顶向下常用桩，自底向上常用驱动。

关键词：Stub 模拟被调用者；Driver 模拟调用者

36. 【名词解释】什么是 4+1 视图？

标准答案：逻辑视图关注功能、类和包；开发视图关注代码模块与构建单元；进程视图关注并发、同步和性能；物理视图关注部署节点与网络；场景视图用关键用例驱动和验证其他四个视图。

关键词：逻辑、开发、进程、物理 + 场景

37. 【名词解释】PO、VO 和 DAO 分别是什么？

标准答案：PO 是持久化对象，通常对应数据库记录；VO 是面向展示或上层传输的值对象，只包含所需数据；DAO 是数据访问对象，封装持久化访问细节并向业务层提供接口。PO 隔离数据库映射，VO 隔离展示需求。

关键词：PO 对持久化；VO 对展示；DAO 封装数据访问

对应材料：复习提纲第 9-10 章；课件《03-02》《03-03》

六、人机交互

38. 【名词解释】什么是可用性？

标准答案：可用性是特定用户在特定环境中使用产品完成特定目标时所达到的有效性、效率和满意度。课程常从易学性、效率、易记性、低错误率/容错性和满意度五个维度理解。

关键词：易学、效率、易记、错误、满意

39. 【简答】至少列出五条界面设计注意事项并解释。

标准答案：保持系统状态可见并及时反馈；使用用户熟悉的语言和现实世界概念；保持术语、布局和操作一致；让选项可见以减少记忆负担；预防错误并提供撤销、恢复；突出主要任务、降低无关信息；兼顾新手提示和专家快捷方式；满足对比度、键盘操作等可访问性要求。

关键词：问题 → 原则 → 用户影响 → 改进

40. 【名词解释】什么是精神模型？

标准答案：精神模型是用户对任务、系统结构和运行方式的理解与预期。界面应贴近用户的任务模型和熟悉概念，而不是暴露数据库、模块或内部实现；模型不匹配会导致学习困难和操作错误。

关键词：用户如何理解系统；界面模型要匹配

41. 【名词解释】什么是用户差异性？

标准答案：用户在经验、年龄、能力、文化、设备和使用环境方面存在差异。设计应支持新手、偶发熟练用户和专家，提供可见提示、合理默认值、快捷方式与可访问性机制，避免只针对“平均用户”。

关键词：用户、设备、环境；新手/偶发/专家

42. 【名词解释】什么是导航？

标准答案：导航帮助用户理解当前在哪里、可以去哪里以及怎样返回。全局导航组织主要任务和主题，局部导航通过布局、标题、面包屑、按钮位置和视觉层次指示当前状态与下一步。

关键词：当前位置、可达位置、返回路径

43. 【名词解释】什么是反馈？

标准答案：反馈是系统对用户操作或自身状态变化给出的可理解响应。简单操作应近乎即时，耗时操作应显示进度或阶段，错误反馈应说明发生了什么、原因和可执行的修复办法。

关键词：及时、明确、可操作

44. 【名词解释】什么是协作式设计？

标准答案：协作式设计强调让用户参与需求、原型和评估，通过观察任务、原型测试和反复修改，使系统适应人的能力与限制。其目标包括低出错率、易记、统一视觉风格和可恢复。

关键词：用户参与、原型、评估、迭代

45. 【简答】如何分析一个界面体现或违反了哪些原则？

标准答案：按“具体问题/优点 → 对应原则 → 对用户的影响 → 改进办法”回答。例如删除与保存按钮相邻且同色，违反错误预防，会增加误触；应拉开距离、降低危险操作视觉权重，并提供确认或撤销。避免只罗列原则名称。

关键词：问题、原则、影响、改进

对应材料：复习提纲第 11 章；课件《03-04-人机交互设计》

七、详细设计与对象协作

46. 【简答】详细设计从哪里出发？

标准答案：详细设计从需求工程和体系结构的结果出发：用例给出功能场景，概念类图给出业务实体，系统顺序图给出系统事件，状态图给出生命周期；架构给出模块规格、导入/导出接口、风格和部署约束。详细设计把这些内容细化到类、接口、方法、协作和算法。

关键词：需求 + 架构 → 类粒度实现蓝图

47. 【名词解释】什么是职责？

标准答案：职责是类或对象应维护的信息或应执行的任务，包括“知道什么”和“做什么”。职责分配应优先考虑信息专家、低耦合、高内聚、创建者和控制器等原则。

关键词：知道什么、做什么

48. 【名词解释】什么是协作？

标准答案：协作是多个对象通过消息传递共同完成较大职责的方式。对象只承担自身合适的职责，把子任务委托给拥有信息或能力的对象；详细顺序图用于表达协作。

关键词：消息、委托、共同完成

49. 【简答】如何进行职责分配？

标准答案：识别系统事件和领域信息；由控制器接收外部事件；把计算与判断交给拥有所需信息的对象；由包含、记录或拥有初始化数据的对象创建组成对象；评估候选方案的耦合、内聚、扩展性和可测试性。

关键词：Controller、Information Expert、Creator、低耦合、高内聚

50. 【简答】集中式、委托式和分散式控制有什么区别？

标准答案：集中式把大部分决策放在少数控制器，流程清楚但控制器易膨胀；委托式由控制器保留主流程并把子任务交给领域对象，通常兼顾清晰与面向对象；分散式把控制进一步散布到许多对象，自治性高但控制流难追踪。

关键词：集中：控制器重；委托：折中；分散：对象自治但难追踪

51. 【名词解释】什么是 Mock Object？

标准答案：Mock Object 是按测试期望模拟真实协作者的测试替身，不仅返回预设数据，还可验证是否以正确参数、次数和顺序发生交互。它用于隔离外部系统、数据库、时间、网络或尚未完成的协作者。

关键词：隔离协作者 + 验证交互

52. 【简答】如何测试类之间的协作？

标准答案：依据详细顺序图确定参与对象、消息和分支；选择被测协作边界；用真实对象或 Mock/Stub 替代外部依赖；验证调用参数、顺序、返回、状态变化和异常传播；覆盖正常流程、替代流程和失败场景。

关键词：顺序图 → 交互测试；正常、分支、失败

对应材料：复习提纲第 12 章；课件《04-01-详细设计》

八、耦合、内聚与信息隐藏

53. 【名词解释】什么是耦合？

标准答案：耦合是模块、类或方法之间相互依赖的程度。耦合越高，一个模块的变化越容易传播到其他模块，理解、测试、复用和修改成本越高；设计通常追求必要且明确的低耦合。

关键词：模块间依赖程度；尽量低但不是零

54. 【名词解释】什么是内聚？

标准答案：内聚是模块内部各职责和元素围绕单一目的结合的程度。高内聚表示模块做一件明确且相关的事，通常更易理解、复用、测试和修改。

关键词：模块内部相关程度；高内聚

55. 【简答】常见耦合类型如何判断？

标准答案：从弱到强可理解为数据耦合、印记耦合、控制耦合、公共耦合、内容耦合。数据耦合只传必要数据；印记耦合传入整个对象但只用一部分；控制耦合通过 flag 控制被调模块逻辑；公共耦合共享全局数据；内容耦合直接依赖或修改对方内部实现。

关键词：数据较好；印记/控制常见；公共/内容较差

56. 【简答】如何改善控制耦合和印记耦合？

标准答案：控制耦合可把分支行为封装为多态或策略，或拆分为职责单一的方法；印记耦合应只传需要的字段、专用参数对象或最小接口，避免让被调用者知道多余数据。

关键词：flag → 多态/策略；大对象 → 必要参数/小接口

57. 【简答】常见内聚类型及改进方向是什么？

标准答案：偶然、逻辑、时间、过程、通信、顺序和功能内聚依次趋强。最佳目标是功能内聚：模块所有元素共同完成一个明确功能。逻辑或时间内聚常把多种行为因“同类”或“同一时间执行”放在一起，可按变化原因和业务职责拆分。

关键词：从偶然到功能；目标是单一明确职责

58. 【名词解释】什么是信息隐藏？

标准答案：信息隐藏是把容易变化、难理解或不应被外部依赖的设计决策封装在模块内部，只通过稳定接口暴露必要能力。它关注隐藏“决策和变化点”，不仅是把字段设为 private。

关键词：隐藏变化决策；稳定接口

59. 【简答】信息隐藏的基本思想和两类常见决策是什么？

标准答案：先识别可能变化的设计决策，再把每个决策分配给单独模块，使外部只依赖抽象契约。常见决策包括：数据表示/数据结构与持久化方式；算法、设备、外部服务或业务规则等实现机制。

关键词：隐藏数据表示；隐藏算法/外部机制

60. 【名词解释】信息隐藏与封装有什么区别？

标准答案：封装是把数据和操作组织到边界内并通过访问控制保护；信息隐藏是设计原则，要求模块边界围绕可能变化的决策建立。封装是实现信息隐藏的重要手段，但有 private 字段不等于隐藏了真正变化点。

关键词：封装是机制；信息隐藏是设计目标

对应材料：复习提纲第 13、15 章；课件《04-02-面向对象的模块化与信息隐藏》

九、模块化与类设计原则

61. 【简答】为什么说全局变量有害？

标准答案：全局变量让多个模块隐式共享可变状态，依赖关系不可见，修改影响范围难判断，并发、测试和复用困难。应缩小作用域，把状态封装进对象，通过参数或明确接口传递，并尽量使用不可变数据。

关键词：隐式依赖、共享可变状态、难测试

62. 【简答】To be Explicit 原则是什么？

标准答案：依赖、数据流、状态变化和副作用应通过参数、返回值、接口和清楚命名显式表达，而不是依赖全局状态、隐含约定或调用顺序。显式设计更易理解、验证和修改。

关键词：依赖显式、契约显式、副作用显式

63. 【简答】Do Not Repeat (DRY) 原则是什么？

标准答案：每一项知识或业务规则在系统中应有唯一、权威的表示。重复代码、常量和规则会导致修改遗漏与不一致；应通过提取方法、抽象模块、配置或共享规则消除“知识重复”，但不要为了表面相似而过早抽象。

关键词：一个知识点一个权威来源

64. 【名词解释】什么是 Programming to Interface / Design by Contract ？

标准答案：面向接口编程要求客户依赖抽象契约而非具体实现；契约式设计用前置条件、后置条件和不变式明确双方责任。这样实现可替换、依赖可注入、错误责任更清楚。

关键词：依赖抽象；前置、后置、不变式

65. 【名词解释】什么是迪米特法则 (LoD) ？

标准答案：对象只应与直接朋友通信，避免穿透对象内部结构的长消息链。这样可以减少对其他对象结构的了解，降低结构变化的传播；可通过委托方法或领域服务封装导航。

关键词：只和直接朋友交谈；避免消息链

66. 【名词解释】什么是接口隔离原则 (ISP) ？

标准答案：客户端不应被迫依赖它不使用的方法。应将臃肿接口拆成面向不同客户角色的小接口，使每个客户只依赖所需能力。

关键词：胖接口拆小；客户不依赖无关方法

67. 【名词解释】什么是里氏替换原则 (LSP) ?

标准答案：任何使用父类型的地方都应能透明替换为子类型而不破坏正确性。子类不能加强前置条件、削弱后置条件、破坏不变式或改变父类承诺的行为语义。

关键词：子类可替换父类；行为契约兼容

68. 【简答】为什么优先组合而非继承？

标准答案：继承造成白盒复用，子类依赖父类内部实现，层次在编译期固定且容易破坏 LSP；组合通过接口委托复用行为，运行时可替换，耦合更低、变化更局部。只有确实满足 is-a 和行为替换时才使用继承。

关键词：继承用于稳定 is-a；组合用于可变行为

69. 【名词解释】什么是单一职责原则 (SRP) ?

标准答案：一个类或模块应该只有一个主要变化原因，即围绕一个明确职责组织。若类同时负责业务计算、持久化、界面和报表，多类变化会共同冲击它，应按变化原因拆分。

关键词：一个变化原因；按职责拆分

70. 【名词解释】什么是开闭原则 (OCP) ?

标准答案：软件实体应对扩展开放、对修改关闭：增加新行为时尽量通过新增实现完成，而不是反复修改稳定代码。常用抽象、接口、组合、多态和策略隔离变化点。

关键词：新增实现，少改稳定核心

71. 【名词解释】什么是依赖倒置原则 (DIP) ?

标准答案：高层策略不依赖低层细节，二者都依赖抽象；抽象不依赖细节，细节实现抽象。通过接口、依赖注入和工厂，使业务逻辑不直接绑定数据库、网络或具体框架。

关键词：高低层都依赖抽象；细节实现抽象

72. 【简答】MyStack extends Vector 是否合理？

标准答案：不合理。Vector 暴露任意位置插入、删除等能力，而栈只承诺压栈、弹栈、大小和判空；MyStack 作为 Vector 子类会暴露并允许破坏栈语义，违反 LSP 和最小接口思想。应使用组合：MyStack 内部持有受控容器，只公开栈接口。

关键词：不是稳定 is-a；组合容器；只暴露栈接口

73. 【简答】Employee extends Person 是否合理？

标准答案：在 Person 的契约仅包含所有雇员都能满足的人员属性和行为时，Employee 是 Person，继承通常合理。若 Person 含 Employee 无法满足的行为或不变式，则仍会违反 LSP。更稳妥的判断标准是行为可替换，而不是只看现实世界名称。

关键词：is-a 只是起点，最终检查行为契约

对应材料：复习提纲第 14-15 章；课件《04-02-面向对象的模块化与信息隐藏》

十、可修改性与设计模式

74. 【简答】如何实现可修改性、可扩展性和灵活性？

标准答案：识别并封装变化点；按职责分解模块；通过稳定接口和依赖倒置隔离实现；使用组合、多态和配置替代大段条件分支；保持低耦合高内聚；用自动测试保护重构。不要为了未知变化过度设计，应围绕可预见变化建立扩展点。

关键词：变化点、抽象、组合、多态、测试

75. 【名词解释】什么是策略模式？

标准答案：策略模式把可互换的算法或业务规则封装为统一策略接口，由上下文组合一个具体策略并委托执行。它可消除按类型增长的 if/switch，使新增算法通过增加策略类实现。

关键词：封装算法族；组合；运行时替换

76. 【名词解释】什么是抽象工厂模式？

标准答案：抽象工厂提供创建一组相互关联或相互依赖产品的接口，而不指定具体类。替换具体工厂即可切换整套产品族，保证同一产品族兼容；缺点是增加新的产品种类需要修改工厂接口及所有具体工厂。

关键词：创建产品族；易换产品族，难加产品种类

77. 【名词解释】什么是单例模式？

标准答案：单例模式保证一个类只有一个实例，并提供全局访问点。适合必须唯一的进程内协调对象，但会引入全局状态、隐藏依赖和测试困难，应谨慎使用，并考虑并发安全与生命周期。

关键词：唯一实例；全局状态风险

78. 【名词解释】什么是迭代器模式？

标准答案：迭代器模式在不暴露集合内部表示的情况下顺序访问元素，把遍历职责从集合中分离。客户端依赖统一遍历接口，集合可更换内部数据结构。

关键词：隐藏集合结构；统一遍历

79. 【简答】遇到设计模式场景时如何答题？

标准答案：先指出变化点和坏味道，如类型分支、创建逻辑散落或遍历暴露；再提出抽象接口与稳定上下文；画出或描述具体实现如何替换；最后说明符合 OCP、DIP、组合优于继承、低耦合等原则及代价。

关键词：变化点 → 抽象 → 结构 → 原则 → 权衡

对应材料：复习提纲第 16 章；课件《设计模式》

十一、软件构造

80. 【简答】软件构造包含哪些活动？

标准答案：详细编码、代码设计、单元测试、调试、代码评审、重构、集成、构建管理、配置管理和构造度量。构造的目标不仅是“能运行”，还包括可理解、可测试、可维护和可集成。

关键词：编码、验证、测试、调试、重构、集成、构建

81. 【名词解释】什么是重构？

标准答案：重构是在不改变软件外部可观察行为的前提下改善内部结构。常见手法包括提取方法、移动方法、拆分、消除重复、用多态替代条件分支；每次小步修改后应运行测试。

关键词：行为不变、结构改善、小步、测试保护

82. 【名词解释】什么是测试驱动开发（TDD）？

标准答案：TDD 按“红-绿-重构”循环：先写一个失败测试，再写最少代码使其通过，最后在测试保护下改进结构。它促进可测试接口、快速反馈和回归保护，但不能替代系统测试和整体设计。

关键词：Red、Green、Refactor

83. 【名词解释】什么是结对编程？

标准答案：两名开发者在同一任务上协作，一人作为驾驶者编写代码，另一人作为领航者审查、思考设计和风险，并定期交换角色。优点是即时评审、知识共享和降低关键人风险，代价是需要良好配合。

关键词：Driver、Navigator、交换角色

84. 【简答】如何提高代码简洁性和可维护性？

标准答案：使用准确命名和单一抽象层次；保持方法短且职责单一；减少参数、共享状态、嵌套和重复；优先清晰的数据结构和标准库；显式处理边界和异常；通过测试、评审、静态检查和重构持续改进。

关键词：命名、短方法、低复杂度、无重复、测试

85. 【简答】如何使用数据结构消减复杂判定？

标准答案：把大量 if/switch 中稳定的“条件-结果”映射提取为表、字典、集合、状态转换表或策略集合，用数据查找代替控制分支。这样规则集中、易扩展、易测试；但表的含义、默认项和非法项必须明确。

关键词：控制逻辑数据化；表驱动

86. 【简答】控制结构和变量使用有哪些原则？

标准答案：优先顺序、选择、循环等清晰结构；减少深层嵌套、复杂布尔表达式和隐式跳转；循环要有清楚边界和退出条件。变量应缩小作用域、初始化后再使用、命名表达含义、避免一变量多用途、优先不可变，并清理魔法数字和全局变量。

关键词：低嵌套、清楚边界；小作用域、单一用途

87. 【简答】语句处理有哪些注意事项？

标准答案：每条语句只表达一个主要动作；保持一致格式；避免副作用隐藏在复杂表达式中；对复杂条件提取命名变量或方法；按正常路径组织代码并尽早返回异常；注释解释“为什么”，而不是重复代码“做什么”。

关键词：一个动作、显式副作用、命名条件、注释原因

88. 【简答】什么是不可维护代码的典型特征？

标准答案：故意含糊的命名、超长方法、巨型类、复制粘贴、魔法数字、全局状态、复杂嵌套、隐式副作用、过多参数、无测试、过期注释和跨层调用。答题时应把坏味道映射到具体重构手法。

关键词：坏味道 + 原则 + 重构

89. 【名词解释】什么是契约式设计 (DbC) ？

标准答案：契约式设计用前置条件规定调用者必须满足的条件，用后置条件规定操作完成后的保证，用不变式规定对象在可观察稳定状态中始终满足的性质。它明确责任边界并支持断言和测试。

关键词：前置条件、后置条件、不变式

90. 【名词解释】什么是防御式编程？

标准答案：防御式编程假设外部输入、文件、网络和第三方系统可能失败，对边界进行校验、设置超时与资源上限、处理异常并保持安全状态。对内部绝不应发生的程序错误可使用断言；不要静默吞掉异常。

关键词：不信任外部输入；校验、超时、异常、安全状态

91. 【名词解释】什么是表驱动方法？

标准答案：表驱动方法把条件与动作、状态转换或计算规则存放在表或映射中，程序通过查表选择行为。它适合规则多、变化频繁且结构相似的逻辑，可减少重复计算和复杂分支。

关键词：规则数据化；查表替代分支

92. 【简答】如何设计单元测试？

标准答案：依据方法契约和代码逻辑覆盖正常、边界、异常和状态变化；遵循 Arrange-Act-Assert；每个测试关注一个行为且可独立、可重复；用 Stub/Mock 隔离外部依赖；测试命名清楚并在修复缺陷时补回归测试。

关键词：AAA；正常、边界、异常；独立可重复

对应材料：复习提纲第 17-18 章；课件《软件构造》《代码设计》

十二、软件测试

93. 【名词解释】什么是黑盒测试和白盒测试？

标准答案：黑盒测试依据需求和接口观察输入输出，不关心内部实现；白盒测试依据代码结构设计数据，关注语句、分支、条件和路径是否被执行。二者互补：黑盒验证功能，白盒发现结构性遗漏。

关键词：黑盒看规格；白盒看结构

94. 【简答】常见黑盒测试方法有哪些？

标准答案：等价类划分把输入分为有效和无效类，每类选代表值；边界值分析选择边界及其附近值；决策表覆盖多条件组合；因果图从输入条件和输出动作推导组合；状态转换测试覆盖合法和非法转换；场景法从用例正常及扩展流程形成测试。

关键词：等价类、边界值、决策表、状态、场景

95. 【简答】语句覆盖、分支覆盖和路径覆盖如何区别？

标准答案：语句覆盖要求每条可执行语句至少执行一次；分支覆盖要求每个判定的真、假分支至少执行一次；路径覆盖要求覆盖控制流中的路径。一般强度为路径覆盖高于分支覆盖，分支覆盖高于语句覆盖，但完整路径覆盖常因循环和组合爆炸而不可行。

关键词：语句 < 分支 < 路径；强度越高成本越大

96. 【名词解释】什么是条件覆盖？

标准答案：条件覆盖要求每个复合判定中的每个基本条件都至少取到一次真和一次假。它不必然保证整个判定的真、假分支均被覆盖，因此常与判定覆盖组合。

关键词：每个基本条件取真/假；不等于分支覆盖

97. 【简答】如何为给定场景选择测试方法？

标准答案：输入范围和分类明显时用等价类；错误易发生在阈值时用边界值；多条件多动作时用决策表；对象生命周期复杂时用状态转换；完整业务流程用场景/用例法；需要检查代码控制结构时使用白盒覆盖。关键系统常组合多种方法。

关键词：根据风险和结构选方法，不机械套一种

98. 【简答】如何由功能需求写测试用例？

标准答案：把每条需求转化为至少一个正常测试和必要的异常/边界测试；写明测试 ID、关联需求、前置条件、输入、步骤、期望输出和后置状态；覆盖权限、业务规则、数据校验和外部系统失败，并保持需求-测试追踪。

关键词：需求追踪；前置、输入、步骤、期望、后置

99. 【简答】如何依据设计图设计集成测试？

标准答案：从包图、类图和顺序图识别模块边界、接口、调用顺序和依赖；优先测试高风险接口和复杂协作；采用增量集成；未完成或不稳定的下层用 Stub，未完成的上层用 Driver；验证数据、异常、顺序和状态一致性。

关键词：设计图 → 依赖图 → 集成顺序 → Stub/Driver

100. 【简答】Mock Object 与 Stub 有什么区别？

标准答案：Stub 主要提供预设返回值，回答“调用后返回什么”；Mock 除返回数据外，还验证“是否按期望方式交互”，如调用次数、参数和顺序。状态验证可偏向 Stub，交互验证常用 Mock。

关键词：Stub 返回；Mock 还验证交互

101. 【简答】JUnit 的基本使用方法是什么？

标准答案：使用测试注解标识测试方法，用断言比较实际值与期望值；用 setup/teardown 准备和清理测试夹具；每个测试独立、可重复；异常使用异常断言；可使用参数化测试覆盖多组数据，并由构建或 CI 自动执行。

关键词：测试注解、断言、夹具、独立、自动化

对应材料：复习提纲第 19 章；课件《软件测试》

十三、软件维护与演化

102. 【简答】为什么软件维护重要？

标准答案：软件交付后仍要修复缺陷、适应环境、增加功能和改善质量，维护通常占生命周期的大部分成本。业务、法规、平台和依赖持续变化，不维护的软件会失去价值或积累安全风险。

关键词：生命周期长、成本高、环境与业务持续变化

103. 【简答】软件维护有哪些类型？

标准答案：纠错性维护修复缺陷；适应性维护适配平台、法规和环境变化；完善性维护增加或改进功能、性能和可用性；预防性维护重构和改善结构以降低未来风险。

关键词：纠错、适应、完善、预防

104. 【简答】如何开发可维护软件？

标准答案：建立清晰架构和模块边界；低耦合高内聚、信息隐藏；使用一致规范和自动测试；文档与代码同步；持续集成、配置管理和可观测性；小步重构、控制技术债；保持需求、设计、代码和测试可追踪。

关键词：结构、测试、文档、配置、重构、追踪

105. 【名词解释】什么是演化式生命周期模型？

标准答案：演化式模型通过多轮版本逐步澄清需求并改进系统，每轮都包含分析、设计、实现和验证。它适合需求不明确或变化频繁的项目，但需要稳定架构、版本控制和持续回归测试。

关键词：多轮版本、反馈驱动、持续演化

106. 【简答】用户文档和系统文档有什么区别？

标准答案：用户文档面向最终用户和运维人员，说明安装、任务操作、错误处理和帮助；系统文档面向开发维护人员，记录需求、架构、设计、接口、数据、部署、测试和决策理由。两者受众、语言和细节层次不同。

关键词：用户文档教怎么用；系统文档解释怎么实现和维护

107. 【名词解释】什么是逆向工程和再工程？

标准答案：逆向工程从现有代码和运行系统中恢复更高层的设计、架构和业务知识，通常不改变系统；再工程在理解旧系统的基础上重构、迁移、重写数据或代码，改善结构和平台并保持或扩展功能。

关键词：逆向工程重在理解；再工程重在改造

对应材料：复习提纲第 20-21 章；课件《软件维护与演化》《软件交付》

十四、软件生命周期与过程模型

108. 【名词解释】什么是软件生命周期模型？

标准答案：软件生命周期模型是对软件从概念、需求、设计、构造、测试、交付、维护到退役各阶段及其顺序、反馈关系和交付方式的抽象。它用于组织工作和风险，不等同于具体工具。

关键词：阶段 + 关系 + 反馈 + 交付方式

109. 【名词解释】什么是 Build-and-Fix 模型？

标准答案：先编码，再根据问题不断修补，缺少系统需求、设计和过程控制。它启动快，适合极小试验，但规模增长后结构退化、成本不可预测、质量和维护风险高，不适合正式大型项目。

关键词：先做后修；小试验可用，正式项目风险高

110. 【名词解释】什么是瀑布模型？

标准答案：需求、设计、实现、测试和维护按阶段顺序推进，阶段边界与文档清晰。适合需求稳定、法规严格、验收明确的项目；缺点是反馈晚、早期错误后期才暴露、适应变化能力弱。

关键词：顺序、文档、稳定需求；反馈晚

111. 【名词解释】什么是迭代模型？

标准答案：把开发分成多轮循环，每轮都进行需求、设计、实现和测试，并利用反馈修正已有成果。它能提前暴露风险、适应变化；代价是需要迭代规划、持续集成和范围控制。

关键词：迭代 = 反复改进同一系统

112. 【名词解释】什么是增量模型？

标准答案：把系统功能分成多个可工作的增量，逐步实现和集成，每个增量增加一部分能力。它可早期交付价值、降低一次性交付风险；但要求架构支持扩展并合理划分增量。

关键词：增量 = 每次增加一部分功能

113. 【名词解释】什么是增量交付？

标准答案：不仅分增量开发，还把每个可用增量交付给用户运行并获取真实反馈。它能更早产生业务价值，但要求部署、兼容、数据迁移和用户变更管理能力。

关键词：增量开发 + 每个增量真实交付

114. 【名词解释】什么是演化式开发？

标准答案：需求和实现通过持续反馈共同演化，初始规格不必完整，系统在多轮版本中趋于成熟。适合创新、需求模糊和变化快的场景；缺点是范围、架构和文档若控制不当会逐渐混乱。

关键词：规格与实现共同演化

115. 【名词解释】什么是原型模型？

标准答案：快速构建可演示原型以澄清需求、验证交互或技术可行性。抛弃式原型用于学习后重做，演化式原型逐步发展为产品；必须避免把质量不足的试验代码直接当正式系统。

关键词：验证需求/技术；抛弃式 vs 演化式

116. 【名词解释】什么是螺旋模型？

标准答案：螺旋模型以风险为驱动，每一圈包括目标与方案确定、风险分析与缓解、开发验证、下一轮计划。适合大型、高风险、需求不确定项目；缺点是管理复杂、成本高且依赖风险分析能力。

关键词：风险驱动；每圈：目标、风险、开发、计划

117. 【简答】迭代增量模型与演化模型有什么差异？

标准答案：迭代增量通常仍有较明确的总体目标和架构，通过多轮改进并逐步增加功能；演化模型允许需求和产品方向在反馈中显著改变。前者更易计划和控制，后者更适应高度不确定，但范围、架构和文档失控风险更高。

关键词：计划性较强 vs 方向可持续演化

118. 【简答】如何根据场景选择过程模型？

标准答案：需求稳定、合规强可选瀑布或阶段门；功能可拆、需要早交付用增量；需求会变化用迭代增量；需求不清、交互重要先做原型；技术和业务风险高用螺旋；持续在线产品通常采用迭代、增量、持续交付组合。答题必须说明场景特征与模型特征的对应。

关键词：场景特征 → 模型特征 → 优点 → 风险

119. 【简答】软件工程知识体系包含哪些主要知识域？

标准答案：核心包括软件需求、软件架构、软件设计、软件构造、软件测试、软件工程运维、软件维护、软件配置管理、软件工程管理、软件工程过程、软件工程模型与方法、软件质量、软件安全、软件工程经济学、软件工程职业实践，以及相关计算、数学和工程基础。考试按课程口径列主要域即可。

关键词：需求、架构、设计、构造、测试、运维/维护、配置、管理、过程、质量、安全

对应材料：复习提纲第 22-23 章；课件《软件开发过程模型》《01-01-软件工程基础》